

Day 5 — Bits as Switches: Masks and Register Fields

Curriculum alignment: Stage 1 — Foundations of Computation · Signed Numbers and Bitwise Reasoning

Target time: ~2 hours (≈45 minutes reading, ≈75 minutes coding)

A hexadecimal register value looks like one number. A driver engineer sees a control panel: one bit enables DMA, another reports an error, and three neighboring bits choose an operating mode.

The surprising bridge: abstract logic became wiring

In the nineteenth century, George Boole developed an algebra of logical statements. Variables could represent alternatives such as true/false, and operations corresponding to AND, OR, and NOT combined them.

For decades this looked more like pure mathematics than a recipe for a computer.

Telephone networks changed the setting. Their engineers already controlled connections with electromechanical relays: current in a coil moved a physical contact, opening or closing another circuit.

In his 1937 MIT master's thesis, Claude Shannon showed systematically that Boolean algebra could describe and simplify relay switching networks. A closed/open circuit could represent a logical alternative; series contacts behaved like AND, parallel paths like OR, and inverted contacts like NOT. Similar ideas were also developed independently, so this is not a single-genius creation story—but Shannon's treatment became enormously influential.

Suddenly an abstract logical expression could become a physical circuit:

```
Boolean expression
  ↓
switching arrangement
  ↓
output voltage/state
```

George Stibitz's relay-based Model K adder, also built in 1937, demonstrated that relay switching could perform binary arithmetic. The missing bridge between "logic on paper" and "calculation in machinery" had become concrete.

The bitwise operations in today's C code are descendants of that bridge. When you write:

```
status & ERROR_MASK
```

you are using Boolean algebra to ask a question about selected physical states. A modern CPU executes it with transistor gates rather than telephone relays, but the logical relationship survives unchanged.

1. Stop treating every bit pattern as one big number

Consider an eight-bit hardware register:

bit:	7	6	5	4	3	2	1	0
meaning:	READY	ERROR	reserved		MODE[2:0]		IRQ_EN	START

The device manual might define:

bit 7	READY	read-only status
bit 6	ERROR	read-only status
bits 5:3	MODE	operating mode, values 0–7
bit 2	reserved	preserve or write zero as documented
bit 1	IRQ_EN	enable interrupts
bit 0	START	begin operation

The register is physically one pattern. Software needs tools that can inspect or modify selected bits without disturbing the rest.

Those tools are bitwise operations.

2. AND: keep only what the mask permits

AND produces 1 only when both input bits are 1:

```
0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1
```

Suppose we want to test bit 6:

```
register: 10100011
mask:     01000000
          ----- &
result:   00000000
```

The result is zero, so bit 6 was clear.

The mask contains 1 only where we want information preserved. Everywhere else, AND forces zero.

In C:

```
if (reg & (1u << 6))
    puts("ERROR is set");
```

The expression `1u << 6` constructs:

```
01000000
```

Clear a bit with AND

To clear bit 1 while preserving everything else:

```
reg &= ~(1u << 1);
```

Reason it out:

```
1u << 1    00000010
~          11111101
reg & mask  preserves every position except bit 1
```

3. OR: force selected bits on

OR produces 1 when either input bit is 1:

```
0 | 0 = 0
0 | 1 = 1
1 | 0 = 1
1 | 1 = 1
```

Set `IRQ_EN` at bit 1:

```
reg |= 1u << 1;
```

Example:

```
register: 10100001
mask:     00000010
          ----- |
result:   10100011
```

Only bit 1 is forced to 1. Every zero in the mask leaves the corresponding register bit unchanged.

4. XOR: toggle selected bits

XOR produces 1 when the inputs differ:

```
0 ^ 0 = 0
0 ^ 1 = 1
1 ^ 0 = 1
1 ^ 1 = 0
```

A mask bit of 1 flips the target; a mask bit of 0 preserves it:

```
reg ^= 1u << 0;
```

Applying the same XOR mask twice restores the original value:

```
x ^ mask ^ mask = x
```

That reversibility makes XOR useful in checksums, cryptographic constructions, difference tracking, and toggles.

5. NOT: invert every represented bit

NOT flips each bit:

```
~00110110 = 11001001    (if the width is exactly eight bits)
```

The width qualification matters. In a C expression, a small integer type is usually promoted to `int` before `~` operates. Therefore:

```
uint8_t x = 0x36;  
printf("%x\n", ~x);
```

may print a wide value such as `ffffffc9`, not merely `c9`.

If you need an eight-bit result:

```
uint8_t inverted = (uint8_t)~x;
```

This is our first glimpse of how C's type rules and hardware width interact.

6. Shifts: move positions or build masks

Left shift:

```
00000001 << 3 = 00001000
```

For unsigned values, shifting left by one position multiplies by two when the result remains representable.

Right shift:

```
00110000 >> 4 = 00000011
```

For unsigned values, shifting right by one divides by two and discards the remainder.

Shifts are not merely arithmetic shortcuts. They position fields.

To create a three-bit mask:

```
unsigned width = 3;  
unsigned mask = (1u << width) - 1u;    /* 00000111 */
```

To move that field to bits 5:3:

```
mask <<= 3;                                /* 00111000 */
```

7. Extracting a field

Our `MODE` field occupies bits 5:3:

```
register: rrrrrrrr
mask:    00111000
```

First clear unrelated bits:

```
reg & 0x38u
```

Then shift the field down to bit 0:

```
unsigned mode = (reg & 0x38u) >> 3;
```

General form:

```
field = (value & MASK) >> SHIFT;
```

Example:

```
register: 10101011
mask:    00111000
AND:     00101000
>> 3:    00000101 → mode 5
```

8. Replacing a field safely

Suppose `reg` already contains status and control bits, and we want `MODE=3`. Blindly OR-ing `3 << 3` is wrong because old mode bits may remain set.

Use two phases:

```
reg &= ~MODE_MASK;           /* clear old field */
reg |= (new_mode << MODE_SHIFT) & MODE_MASK;
```

Why mask after shifting?

If `new_mode` accidentally exceeds three bits, masking prevents it from leaking into neighboring fields. Production code should usually validate the input as well.

Many kernel helpers package this pattern:

```
FIELD_PREP(MASK, value)
FIELD_GET(MASK, register)
```

We will use real kernel definitions later. For now, understand the operations they hide.

9. Reserved bits are not spare bits

A device manual may label some bits “reserved.” That does not mean software may use them.

Depending on the interface, reserved bits might need to be:

- written as zero,

- preserved with read-modify-write,
- ignored when read,
- or handled according to a future hardware revision.

This makes the difference between these operations important:

```
reg = IRQ_EN;      /* replaces every bit: potentially destructive */
reg |= IRQ_EN;     /* sets one bit, preserves others */
```

Read the specification. Bit operations are mechanisms; register semantics are policy defined by the hardware contract.

10. Hands-on C challenge

Open:

```
~/masterclass/daily/challenges/day-005-register-mask-lab.c
```

It contains a fictional eight-bit device register and tests for functions that:

1. set a bit,
2. clear a bit,
3. toggle a bit,
4. test a bit,
5. extract `MODE`,
6. replace `MODE` without corrupting neighboring bits.

Build it:

```
gcc -std=c17 -Wall -Wextra -Wconversion -O0 -g \
~/masterclass/daily/challenges/day-005-register-mask-lab.c \
-o /tmp/day-005-register-mask-lab

/tmp/day-005-register-mask-lab
```

The initial file deliberately contains TODO implementations. Predict each mask on paper before writing C.

11. Read a real Linux-style value

CPU feature sets, file modes, signal sets, capabilities, and device registers all use bits as independent flags or packed fields.

Try:

```
stat -c 'mode in hex-like form: %f permissions: %A' /bin/ls
```

The mode value packs file type and permission flags into one integer. Later, when studying files, we will decode its exact fields. For now notice the pattern: a “number” is serving as a compact collection of switches.

You can also inspect kernel bit macros:

```
rg '#define BIT' /usr/src/kernels/*/include/linux/bits.h
```

This deliberately searches the installed development kernel because it may not yet match the running kernel. The important idea is:

```
BIT(n) → one bit positioned at n
```

12. Checkpoint

Given:

```
register = 10101101
```

Explain and calculate:

1. Is bit 6 set?
2. What mask sets bit 1?
3. What mask clears bit 0?
4. What value is stored in bits 5:3?
5. How would you replace bits 5:3 with `010` while preserving every other bit?

Do not answer with only C syntax. Draw the register, mask, and result.

References used selectively

- Patt and Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 2, logical operations and bit vectors (book pages 33–36).
- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 2, §2.1 on information storage and bit-level operations.
- Current Linux kernel headers, particularly `include/linux/bits.h` and `include/linux/bitfield.h`, for the modern shape of `BIT`, `GENMASK`, `FIELD_GET`, and `FIELD_PREP`.
- Computer History Museum, "How Do Digital Computers 'Think'?" for Boole, Shannon's relay-circuit work, and switching logic: <https://www.computerhistory.org/revolution/digital-logic/12/269>
- Computer History Museum, 1937 timeline entry for George Stibitz's Model K relay adder: <https://www.computerhistory.org/timeline/1937/>

Next: Day 6 asks what happens when arithmetic produces a mathematical answer that does not fit: unsigned wraparound, signed overflow, carry, overflow flags, and sign extension.